

PungentFunk Utilities

Architecture and Design Principles Bible

Consistency Contracts, Developer Mode, Shared Chrome, Package Modularity, Authoring Foundation, and Overlay Tray Doctrine Edition

Updated: 2026-05-12

This edition refines the 2026-05-10 package, authoring, and overlay doctrine using the current source archive and the requested Pro-mode consistency pass. It is source-informed doctrine, not a claim that every active Unity UI state has already been manually validated.

Purpose

Provide a stable, enforceable doctrine source for PungentFunk Utilities: architecture, package boundaries, shared UI contracts, Developer Mode gating, window chrome, minimizer/tray behavior, overlay trays, source-truth verification, authoring data contracts, shared scan systems, optional dependency behavior, AI-agent interpretation, and release-hardening rules.

Central refinement in this edition

The previous doctrine was directionally strong. This edition makes the shared principles enforceable. Every utility should consume common contracts for registry metadata, Developer Mode access, shared window chrome, minimize/tray behavior, help handoff, package status, overlay trays, shared scans, and optional package fallbacks instead of reinterpreting those behaviors locally.

Contents

- 0. How to use this bible
- 1. Scope and North Star
- 2. Product Architecture
- 3. Shared Utility Consistency Contracts
- 4. Developer Mode Root-Gate Contract
- 5. Shared Window Chrome, Minimize, Help, and Package Badge Contract
- 6. Package Modularity and Extension Ownership Doctrine
- 7. Registry, Descriptor Metadata, and Per-Package Registrar Doctrine
- 8. Package Presence, Missing Dependency, and Simulation Doctrine
- 9. Shared Authoring Data Foundation Doctrine
- 10. Source-Truth and Verifiability Doctrine
- 11. UI Preservation and No-Hiding Doctrine
- 12. Editor UI and UX Principles
- 13. Window and Panel Layout Doctrine
- 14. Overlay Tray and Popup Surface Doctrine
- 15. Progressive Disclosure, Assistance, Accessibility, and Messaging
- 16. Unity Editor Implementation Doctrine
- 17. Safety and Performance Doctrine
- 18. Shared Package Systems and Project Scan Pipeline Doctrine
- 19. Editor Access and Integration Principles
- 20. Optional Dependency and Cross-Utility Integration Principles
- 21. Runtime, Editor, Data, and Serialization Rules
- 22. Asset, Package, and Import Pipeline Doctrine
- 23. Package Split Map
- 24. Lab Taxonomy
- 25. Documentation Architecture
- 26. AI Task Routing and Failure Prevention Matrix
- 27. AI Output, Briefing, and Debrief Standards
- 28. Package Hardening Rules
- 29. Versioning, Review, and Maintenance
- 30. Utility Definition of Done
- 31. UI Refactor Definition of Done
- 32. Consistency Acceptance Grid
- 33. Authoring and Package Split Review Gates
- 34. Reference Basis
- 35. Closing Doctrine

0. How to use this bible

What this document is

This is the doctrine source for PungentFunk Utilities. It records durable architecture and package boundaries, Unity Editor implementation doctrine, shared UI consistency rules, accessibility and messaging principles, dependency and optional integration rules, Developer Mode rules, overlay tray/windowing doctrine, source-truth and verification standards, shared scan pipeline doctrine, authoring data contracts, AI-agent interpretation rules, and release-hardening expectations.

It does not record volatile script counts, sprint tasks, one-off bug lists, per-PR implementation deltas, exhaustive current implementation inventory, tutorial-level API steps, or complete code examples. Those belong in snapshot audits, pattern pages, lab specs, issue briefs, and implementation notes.

Reading paths

Reader	Read first	Then read
AI coding agent preparing an audit	Sections 0, 2, 3, 4, 7, 8, 10, 11, 18, 26, 27, 30, 31, 32, 33	Current source files, current snapshot audit, relevant package map, and active UI evidence.
AI coding agent preparing an implementation brief	Sections 0, 2, 3, 4, 5, 7, 8, 13, 14, 16, 17, 18, 19, 27, 28, 30, 31, 32	Relevant lab spec, current source tree, path-specific instructions, package ownership rules, validation grid.
AI coding agent editing UI code	Sections 3, 4, 5, 10, 11, 12, 13, 14, 15, 31, 32	Current active UI path, control inventory, shared chrome/tray helpers, developer gate helpers, and validation steps.
Human implementer	Sections 1, 2, 3, 4, 5, 6, 7, 12, 13, 16, 17, 18, 19, 21, 23	Relevant lab spec, implementation pattern page, package manifest, and current source.
Reviewer	Sections 3, 4, 5, 7, 8, 10, 11, 14, 17, 18, 28, 30, 31, 32, 33	Changed files, active UI, package install matrix, compile logs, validation evidence.
Designer or UX contributor	Sections 3, 5, 11, 12, 13, 14, 15, 19, 31, 32	Visual examples, layout patterns, shared component catalogue, help/documentation patterns.
Producer or planner	Sections 1, 2, 6, 7, 8, 23, 24, 25, 29, 33, 34, 35	Roadmap, package release plan, snapshot audits, decision log.

Source-of-truth hierarchy

1. The user current explicit instruction.
2. Current uploaded source files or current repository source tree.
3. The active UI as rendered in Unity, including visibility conditions, layout, scroll behavior, resizing behavior, focus behavior, and interactive reachability.
4. This design bible for durable doctrine.
5. The current snapshot audit for implementation state.
6. The current feature inventory for backlog/status.
7. Lab specs and pattern pages for detailed implementation guidance.
8. Historical briefs, older audits, old debriefs, and conversation notes.

A debrief, generated summary, or changelog entry is context, not proof. Source reality and active UI reality win.

Implementation-state vocabulary

Term	Meaning
Implemented in source	Relevant classes, fields, methods, registrations, provider hooks, and call paths exist in current files.
Reachable in UI	A user can access the feature through the intended menu, window, browser card, inspector, context menu, overlay, tray, or shortcut without hidden prerequisites.
Visible in UI	The feature is displayed at practical sizes and is not obscured, clipped, buried, hidden behind an accidental layout expansion, or only exposed by stale duplicate UI.
Usable in UI	Controls are legible, interactable, not cramped beyond reasonable use, and give expected feedback.
Verified	Source evidence, active call path, active UI evidence, and validation

Term	Meaning
	steps support the claim. Do not call a UI feature complete without all of these.

Normative words

Word	Meaning
Must	Required unless the user explicitly overrides the rule and accepts the risk.
Should	Default expectation. A deviation needs a stated reason.
Prefer	Strong recommendation; alternatives are acceptable when the workflow demands them.
Avoid	Do not do this without a documented reason.
Forbidden	Do not implement this in generic PFU packages.
Experimental	Allowed only when marked, reversible, documented, and isolated from required user workflows.

1. Scope and North Star

PungentFunk Utilities is a family of modular Unity editor/runtime utility assets, not a monolithic toolbox. Each lab or suite should be independently releasable while the larger bundle exposes a cohesive control panel, shared visual language, consistent interaction vocabulary, and optional cross-lab enhancements.

The core promise is simple: automate tedious game-development workflows without adding new tedium. Every tool should be understandable at a glance, safe to try, powerful enough to scale through presets and profiles, and integrated into the Unity Editor surface where the user naturally needs it.

A tool that technically exists but is hidden, inaccessible, cramped, duplicated, compile-fragile, package-fragile, ungated, or visually degraded has not met the product promise.

Release model

Tier	Role	Rule
Free/Core package	Registry, launcher, theme, preferences, shared layout helpers, shared window chrome, minimizer/tray, help opening contracts, shared scan/result contracts, provider registries, Developer Mode contracts, token parser contracts, package presence contracts, and Authoring Data Foundation contracts.	Keep Core small, stable, conservative, and boring. Core may host browser shells and fallback UI, but must not ship every concrete editor workflow.
Standalone extension packages	Independently useful packages: Documentation and Authoring, Board/Graph, Data Sheet, Audit/Validation, Scene Authoring, Asset Production, Appearance/Colour/Texture, Audio, Debug, UI/Input Feedback, and Content Generation.	Extensions own concrete workflows and may depend on Core. They must not assume unrelated extensions exist.
Bridge packages	Optional glue connecting two PFU packages or a PFU package and a third-party package.	Bridge packages must be removable, version-safe, and gracefully degraded when absent.
Full Bundle package	Installs all extensions and official bridges; enables richer dashboards, handoffs, cross-lab context menus, and coordinated workflows.	Bundle success is not sufficient. Standalone configurations must compile and run.
Project adapters	Thin game-specific wrappers, presets, aliases, and translators outside generic packages.	They may reference project classes. Generic packages must not.

Definition of a PungentFunk utility

- Solves a broadly reusable Unity workflow problem.
- Uses generic names, generic data models, stable IDs, and configurable presets.

- Works in a clean project or clearly declares optional dependencies.
- Provides preview or dry-run before destructive changes and records Undo where scene or asset changes are made.
- Uses shared visual language, shared chrome, shared Developer Mode gates, shared minimize/tray behavior, and shared persistence helpers unless deliberately exempt.
- Chooses an appropriate Unity Editor integration point rather than forcing every workflow into one window.
- Remains visible, reachable, usable, and verified after refactors and cleanup passes.
- Declares package ownership, dependencies, capabilities, extension points, bridge state, missing-provider fallback behavior, documentation topic, and UI access surfaces.

2. Product Architecture

Package layers

Layer	Purpose	Rules
Core	Shared shell, registry, browser, menus, theme, preferences, shared UI contracts, minimizer/tray, package metadata contracts, presence resolver, shared scan/result models, documentation patterns, design tokens, provider registries, Developer Mode gates, and Authoring Data Foundation contracts.	No Core-to-lab dependencies. Keep public APIs conservative. Core may own interfaces, registry/service lookup, shared UI helpers, and fallback states, not concrete lab workflows.
Runtime model assemblies	ScriptableObjects, MonoBehaviours, data models, adapters, and runtime services used outside the Unity Editor.	No UnityEditor references. No editor-only menu logic. Use stable IDs, version fields, migration-safe data, and player-safe defaults.
Editor lab assemblies	Editor windows, inspectors, scanners, generators, validators, TreeViews, Scene GUI helpers, overlays, graph tools, authoring panels.	May depend on UnityEditor and Core. Scan/apply operations must be explicit, cancelable where expensive, and cached. Drawing code must not own mutation logic.
Optional bridges	Code connecting labs or third-party packages.	Use version defines, reflection-safe discovery, asmdef constraints, isolated bridge packages, package metadata, or registry/service discovery.
Project adapters	Game-specific wrappers and presets outside generic packages.	Thin only. Generic packages must never compile against them.

Dependency direction

Allowed	Discouraged	Forbidden
Lab -> Core; Editor -> Runtime; Project Adapter -> Generic Lab; Bridge -> Lab A + Lab B.	Lab A directly compiling against Lab B unless Lab B is declared required for that product.	Core -> Lab; Runtime -> Editor; Generic Lab -> game-specific class; cyclic lab dependencies; accidental compile-time dependencies on optional labs.

Core Anti-Sink Rule

Core exists to reduce duplication, not to absorb product features. Core may own stable contracts, service lookup, shared state vocabulary, provider registries, descriptors, package presence, fallback states, shared window chrome, shared tray manager, and simple shell UI. Core must not own concrete lab editors, domain-specific scanner logic, rich document editing, board editing, sheet editing, audit provider implementations, scene placement logic, colour/audio/debug workflows, or package-specific generation behavior.

Decision test

If code interprets domain-specific meaning, generates domain-specific assets, scans domain-specific content, mutates domain-specific records, or renders a concrete editor workflow, it belongs in a lab or bridge. If it defines stable identity, metadata, provider contracts, shared UI/service vocabulary, gating, or safe fallback state, it can belong in Core.

3. Shared Utility Consistency Contracts

This section is the main refinement in this edition. It converts shared principles into contracts that every utility must consume unless the utility documents a justified exception.

Updated 2026-05-12 - Consistency Contracts, Developer Mode, Shared Chrome, Package Modularity, Authoring Foundation, and Overlay Tray Doctrine

Shared contract rule

A utility may have a unique workflow, but it must not invent its own behavior for global concerns. Registry identity, access, Developer Mode, package state, help handoff, minimize/tray, status badges, overlay trays, scan freshness, empty states, and common toolbar/filter behavior must use shared contracts.

Consistency contract catalogue

Contract	Core owner	Required behavior
Developer Mode gates	Core DeveloperMode	One root switch in Utilities Browser -> Browser Settings -> Developer Tools. No developer-only control appears or functions unless the relevant gate is true.
Utility window chrome	Core UI	Common title/status/help/package/developer/minimize affordances for all eligible utility windows.
Minimize and tray	Core minimizer/tray	Same action semantics, tooltip, icon/label pattern, tray state, restore behavior, and persistence across utilities.
Package status	Core package contracts	Every utility card and package-gated surface explains ownership, installation, optional dependencies, and missing-package fallback.
Help/context handoff	Core help contracts	Help affordances preserve utility ID, package ID, tab/module, section, selected item, and topic anchor.
Overlay trays	Core tray manager	Lightweight popup-style surfaces use shared tray requests, placement, clamping, focus, persistence, and full-tool handoffs.
Search/filter rows	Core UI patterns	Rows wrap by semantic groups, preserve filter state, and avoid horizontal scroll except for inherently horizontal data.
Scan/finding UI	Core scan contracts	Project-wide facts use shared provider/result/cache/finding/action vocabulary before local scanners.
Authoring metadata	Core Authoring Data Foundation	Documents, notes, boards, sheets, tokens, help, audit issues, utilities, and external targets share stable IDs, links, metadata, preview, actions, and validation results.

Shared component catalogue

Component	Used by	Rule
Utility window chrome	Every eligible utility window	Title, subtitle/status, package badge, help, minimize/tray, and Developer Mode badge are centralized.
Browser settings panel	Utilities Browser	Owns global display settings and the Developer Tools root toggle.
Developer gate card	Any developer surface	Explains why controls are hidden/disabled and where to enable them.
Package status card	Browsers, trays, missing-extension surfaces	Shows package owner, installed/missing state, capability, install hint, and safe fallback action.
Utility card/action row	Browser, hubs, trays, quick launch	Uses registry descriptor metadata and common button/status patterns.
Overlay tray surface	Help previews, filters, recent tools, selected-item summaries	Uses shared tray manager, bounds, focus, persistence, and handoff rules.
Minimize/tray control	Window chrome, toolbars, commands	Routes through Core minimizer/tray; local parked-window behavior is forbidden.
Help/context button	Windows, rows, inspectors, overlays	Carries full context and opens the appropriate help/browser/tray.
Search/filter row	Browsers, scanners, registries, dense lists	Wraps by semantic groups and persists text/filter state.

Component	Used by	Rule
Empty state card	All no-data/no-selection/no-package states	Explains what is missing and offers one safe next action.
Finding/result row	Audit, validation, token scans, coverage tools	Shows severity, target, package/provider, freshness, primary action, and details.
Split panel	Two/three-panel windows	Uses shared splitter math, clamping, persistence, and validation.
Open Full Tool handoff	Trays, inspectors, mini panels	Required when compact content can grow into a full workflow.

Local exception rule

A utility may bypass a shared helper only when the task has a genuine canvas, graph, matrix, timeline, or Scene View interaction requirement. The exception must state which shared helper was bypassed, why it was insufficient, what equivalent behavior remains, and how the utility validates narrow/wide, focus, persistence, and optional package states.

4. Developer Mode Root-Gate Contract

Developer Mode is a global root gate, not a per-window preference. The root switch lives in Utilities Browser -> Browser Settings -> Developer Tools. It is the only user-facing switch that enables developer controls across the suite.

Gate levels

Gate	Source API / concept	Allows
Visibility gate	PungentDeveloperMode.Enabled	Shows Developer Mode UI, developer filters, internal utility cards, archived/hidden filters, diagnostics, and explanatory developer badges.
Metadata mutation gate	PungentDeveloperMode.CanEditProjectMeta data	Allows editing utility metadata, package catalog overrides, documentation links, category membership, hidden/internal/archive flags, developer-only flags, and authoring metadata.
Source authoring gate	PungentDeveloperMode.CanEditPresetSourc es plus internal define	Allows source-writing preset tools, generated source stubs, package-facing preset code, or other source-producing tools.

Absolute gating rule

No developer-only control may appear, enable, mutate data, run actions, open source-authoring tools, write metadata, bypass package/status gates, or become callable from menus, context menus, overlays, trays, inspectors, editor windows, shortcuts, browser cards, help panels, or stored callbacks unless the relevant Developer Mode gate is true.

Developer Mode controls include

- Utility metadata editors and project-local overrides.
- Package catalog overrides and package installation simulation.
- Hidden, internal, archived, developer-only, generated, and experimental filters when those states are not normal user workflow.
- Documentation link editing, generated documentation controls, topic stub generation, and source/doc index mutation.
- Category membership, facet editing, menu taxonomy authoring, and utility-card metadata editing.
- Source-writing preset tools, especially theme preset source authoring and package-facing generator code.
- Diagnostic mutation panels, design validation internal actions, and developer-only package simulation.
- Any toolbar, row, inspector, tray, overlay, popup, shortcut, or context menu action that writes developer-facing metadata or bypasses user-facing state.

When Developer Mode is disabled

- Developer controls are not drawn.
- Developer actions do not execute even if called by direct menu, stored shortcut, tray state, reflection, or stale callback.
- Read-only badges are allowed when useful: Developer Only, Archived, Generated, Internal, Missing Package, Experimental.

- A disabled explanatory card may appear only when the card helps the user understand why a documented surface is unavailable.
- Normal workflows must not require Developer Mode. If a normal user needs the control, it is not developer-only.

When Developer Mode is enabled

- Developer surfaces appear in predictable, labeled places.
- Each developer control states whether it changes project-local data, package metadata, generated documentation, or source files.
- Dangerous or source-writing developer controls remain separately gated by source-authoring availability.
- Developer badges must not use color alone. Pair color with text labels and tooltips.
- Developer filters must leave visible filter state and a way to restore default browsing.

Required integration points

Surface	Developer Mode behavior
Utilities Browser settings	Only root toggle location. It also owns explanatory warning text and root state persistence.
Utilities Browser filters	Developer filters appear only when the root gate is enabled.
Utility cards/actions	Developer-only edit/link/package override actions appear only when the root gate and mutation gate are satisfied.
Help Browser	Developer-only topics and generated coverage filters appear only through explicit Developer Mode filters.
Authoring Browser / Notes / Boards / Sheets	Mutable archived/developer-only metadata controls use the metadata mutation gate. Read-only badges may remain visible.
Theme preset authoring	Source-writing preset controls require the source authoring gate, not only the visibility gate.
Overlay trays and popup surfaces	Developer controls in trays follow the same gates and must not remain callable after the root gate is disabled.
Menus and shortcuts	Direct developer menu paths either stay hidden/disabled or open a gate card when Developer Mode is disabled; they must not perform the action.

Developer tool catalogue

Tool family	What it offers	Access doctrine
Registry metadata editor	Utility card metadata, labels, documentation topic IDs, visibility, package IDs, category facets, and local overrides.	Accessible only through Developer Mode browser actions or direct gated command. Requires metadata mutation gate.
Category reference / membership	Stable category IDs, display overrides, utility category facets, taxonomy validation.	Developer Mode only. Changes are project-local unless explicitly source-authored.
Documentation link editor	Assign current/backlog documentation links, validate targets, manage generated/archived links.	Safe read/open stays user-facing. Editing requires Developer Mode or the Documentation and Authoring package authoring role.
Package catalog / simulation	Package records, display names, capability lists, install hints, developer override availability.	Developer Mode only. Real installed-package state must remain separate from simulated state.
Design validation audit	Architecture, registry, menu, documentation, source/UI, package, and layout diagnostics.	May be visible as an audit utility, but mutation, package simulation, and internal diagnostics require Developer Mode.
Theme preset source authoring	Generate, archive, or update source-backed theme preset entries.	Requires source authoring gate and internal source-authoring define.
Debug diagnostics	Debug routing, signal/channel diagnostics, reflected component state panels, scheduled debug actions.	Debug package may be product-facing, but developer-only controls within it still require Developer Mode.

5. Shared Window Chrome, Minimize, Help, and Package Badge Contract

Every eligible PFU utility window should use shared window chrome for the utility-level actions and status that users need before they enter the unique workflow of the tool.

Required window chrome elements

Element	Required behavior
Title and subtitle/status	Clear utility name, short workflow promise, current package/status where useful, and current selection/context when relevant.
Package badge	Uses descriptor package metadata and presence resolver. Shows missing/optional/bundle/developer states consistently.
Help button	Carries utility ID, package ID, module/tab, section, selected item, and topic anchor. Opens context help or a help tray/browser handoff.
Minimize to Tray control	Routes through the Core minimizer/tray service. Same label, icon, tooltip, hit target, and state semantics.
Developer badge	Appears only when Developer Mode is enabled and the current surface contains developer-only controls or data.
Primary action group	Shows the main scan/preview/apply/create action near the header when the workflow requires persistent access.
Status/freshness row	Shows cached scan age, selection state, package state, or missing dependency state where relevant.

Minimize and tray semantics

- The shared command is an action, not a local toggle, unless a specific panel is truly being collapsed in place.
- Use the phrase Minimize to Tray for full window minimization. Compact icon-only variants require a tooltip with the same action name.
- Do not create local parked-window systems, utility-specific minimize toggles, or duplicate tray stacks.
- All minimized utility state routes through the Core minimizer/tray service.
- Restoring a minimized utility returns the user to the previous utility/window state as safely as Unity permits.
- Minimized items must show utility name, package badge where useful, recent status, restore action, and clear/remove action.
- Developer-only minimized entries must not reveal developer controls unless Developer Mode is enabled.

Help and context handoff

- Help affordances preserve context: utility ID, package ID, tab/module, section, selected row/item, selected scene/object/asset, and topic anchor.
- Compact surfaces should open a contextual help tray first when the user benefits from staying in place.
- Long help, generated docs, troubleshooting, and reference material open the Help Browser or a full documentation window.
- Developer-only help topics and generated docs are hidden or filtered unless Developer Mode is enabled.

Shared chrome implementation target

Candidate source contract	Responsibility
PungentUtilityWindowChrome	Draw or create standard header/chrome for IMGUI and provide equivalent data for UI Toolkit.
PungentUtilityWindowChromeOptions	Title, subtitle, package ID, utility ID, status, selection context, help topic, minimize eligibility, developer badge, primary actions.
PungentUtilityMinimizeButton	Common minimize button rendering and routing.
PungentUtilityHelpContext	Context object passed to help browser/tray/documentation links.
PungentUtilityPackageBadge	Package presence, install hint, bridge state, and missing dependency chip/card.

6. Package Modularity and Extension Ownership Doctrine

PungentFunk Utilities must support modular optional extension packages. The free/Core package should be useful and stable, but it should not force every advanced editor or lab workflow into Core. The full bundle should install and coordinate all extensions, but no extension should rely on the full bundle unless it explicitly declares itself bundle-only.

Distribution tiers

Tier	Owns	Must not own
Free/Core	Stable contracts, registries, shared theme/preferences, shared chrome, minimizer/tray, minimal browser shells, shared scan and authoring contracts.	Concrete lab workflows, advanced editors, or paid-domain implementations.
Paid extension	Concrete lab UI, workflows, domain models, scanners, generators, import/export, templates, panels, and domain provider registration.	Core-only services or direct dependencies on unrelated optional packages.
Bridge	Cross-package handoffs and optional integration glue.	Primary ownership of either side workflow.
Full Bundle	Packaging, samples, all official bridges, bundle landing pages, cross-lab dashboards.	Hidden dependency assumptions that make standalone packages fragile.
Project adapter	Game-specific presets, wrappers, aliases, compatibility translation.	Generic package logic or assumptions.

Package capability declarations

Every significant utility, provider, bridge, authoring item provider, scan provider, overlay surface, and menu action should declare package ownership and capabilities. This allows the Utilities Browser, Authoring Browser, Help Browser, Developer Mode filters, package cards, missing dependency UI, and optional bridge UI to explain what is installed, missing, and provided.

- packageId and packageDisplayName.
- packageTier: Core, Extension, Bridge, Bundle, ProjectAdapter, Internal.
- packageOwner and moduleId.
- requiredPackageIds and optionalPackageIds.
- providedCapabilities and consumedCapabilities.
- extensionPointsProvided and extensionPointsConsumed.
- bridgeId and bridgeAvailabilityState.
- fallbackBehavior and missingDependencyMessage.
- installHint and minimumCompatibleVersion.
- isDeveloperOnly, isExperimental, and isBundleOnly.
- documentationTopicId and relatedUtilityIds.

Bridge availability states

- InstalledAndEnabled
- InstalledButDisabled
- MissingRequiredPackage
- VersionMismatch
- MissingOptionalProvider
- DeveloperModeOnly
- UnsupportedInThisContext

Extension feature gating

Feature gates should be visible, calm, and explanatory. A missing optional package must not compile-break, throw editor exceptions, leave dead buttons, or hide user data. Unsupported actions should be disabled with a concise reason and, where appropriate, an install, enablement, package, or bridge hint.

Future brief rule

Every brief that adds a utility, provider, editor, scanner, data type, bridge, overlay, tray, help topic, package card, or menu entry must state which package owns it, which dependencies are required, which dependencies are optional, which capabilities it provides, which extension points it consumes, how Developer Mode applies, and how the UI behaves when optional packages are missing.

7. Registry, Descriptor Metadata, and Per-Package Registrar Doctrine

The registry is the package-level discovery contract. The Core registry owns registration APIs, descriptors, lookup, filtering, sorting, launch policy, recent/favourite/pinned state, and fallback states. It must not own concrete extension registrations as a long-term architecture.

Registry descriptor contract

Field group	Required content
Identity	Stable kebab-case ID, display name, description, lab/category/module, menu/access path, related utility IDs. Never reuse an ID for a different tool.
Implementation role	EditorWindow, CustomEditor, PropertyDrawer, TreeView, Scene GUI, Overlay, Runtime Service, Bridge, Graph Tool, Hybrid, Action, Provider, or Tray.
Capabilities	Selection, context menu, scene overlay, batch action, lab hub, tree data, scene handles, inspector embedding, graph workspace, search, shortcut, accessibility metadata, package bridge, tray surface.
Package metadata	Package ID, tier, required/optional packages, provided/consumed capabilities, extension points, bridge state, missing dependency message, install hint, documentation topic.
Visibility	Visible, Hidden, DeveloperOnly, Archived, Internal, Experimental, BundleOnly, MissingPackage, Deprecated.
Fallback behavior	Disabled state, read-only preview, copy/open raw, package hint, install action, bridge hint, or full-tool handoff.

Descriptor metadata saturation rule

A utility registration is incomplete unless its descriptor explicitly resolves package ID, package tier, implementation role, primary access path, capability flags, visibility model, required packages, optional packages, documentation topic, and missing-dependency behavior before it appears in user-facing browsers.

Per-package registrar contract

Core must expose registration APIs and discovery services. Each package must provide a registrar that registers only its own utilities, actions, providers, menus, overlays, trays, shortcuts, and optional bridge hooks. Core must not import concrete extension window namespaces to register them by default.

Registrar	Registers
PungentCoreUtilityRegistrar	Utilities Browser, theme customizer, tray/minimizer, help browser read/open, core package cards, Developer Mode contracts, shared scan contracts, Authoring Browser shell.
PungentAuthoringUtilityRegistrar	Rich Document Editor, advanced Authoring Browser, legacy Notes manager, documentation-link editing, templates, token insertion helpers.
PungentBoardUtilityRegistrar	Board/Whiteboard editor, graph editor, relationship maps, minimap, board previews.
PungentDataSheetUtilityRegistrar	Data Sheet Editor, CSV import/export, scanner-generated sheet views, Coverage Matrix successor.
PungentAuditUtilityRegistrar	Design Validation Audit advanced UI, reference scanners, terrain scanners, token validator advanced scan UI, release readiness, findings actions.
PungentSceneAuthoringUtilityRegistrar	Asset Placement Lab, Surface Align, Modular Path Builder, Scene Navigation, Scene Gizmo Browser, Area Authoring Toolkit, placement/path actions.
PungentAssetProductionUtilityRegistrar	Prefab Icon Generator, Prefab Asset Exporter, Font Preview, Texture Array Baker, preview/export queues.
PungentAppearanceUtilityRegistrar	Palette Designer, colour analysis/accessibility, theme preview integrations, procedural texture workflows.
PungentAudioUtilityRegistrar	Audio Setup Coverage, Audio Catalog Coverage, audio profile/matrix editors, audio validation providers.
PungentDebugUtilityRegistrar	Debug Controller, DebugRouter panels, scheduled actions,

Registrar	Registers
	diagnostics, reflected state panels.
PungentUIInputUtilityRegistrar	Input Prompt Icon Library, input prompt/token bridges, UI feedback presenters.
PungentContentGenerationUtilityRegistrar	Name Generator, name-list tools, MadLib tools, text generation helpers.

Core-to-extension source review rule

A Core source file may reference Core, shared contracts, fallback UI, and registrar discovery only. If a Core file imports a lab namespace, the pass must justify it as a temporary compatibility bridge, move it to a package registrar, or mark it as a release blocker.

Registry validation

- No missing packageId.
- No missing implementation role for product-facing utilities.
- No missing documentationTopicId for product-facing windows and actions.
- No developer-only utility visible by default.
- No extension utility registered by Core default registration.
- No package-gated utility without fallback/missingDependencyMessage.
- No stale legacy alias pointing to a competing full implementation.
- No direct action that bypasses Developer Mode gates or package presence gates.

8. Package Presence, Missing Dependency, and Simulation Doctrine

Package ownership is not the same as package presence. A package map says which package owns a utility. Descriptor metadata says what the utility provides and consumes. A presence resolver says what is actually installed in the current project. Developer simulation overrides test missing-package states, but they must be visually labeled and never confused with real installation state.

Separation of concerns

Concept	Purpose	Must not do
Package map	Defines target ownership and release planning.	Must not be treated as proof that a package is installed.
Descriptor metadata	Defines package ID, tier, dependencies, capabilities, visibility, fallback behavior.	Must not silently hide missing dependencies.
Package presence resolver	Checks actual installed packages, asmdefs, providers, symbols, and optional bridge availability.	Must not rely only on default catalog records.
Developer simulation override	Temporarily simulates installed/missing/disabled states for validation.	Must not appear as real package state or function when Developer Mode is disabled.

Recommended package presence contracts

Contract	Responsibility
IPungentPackagePresenceProvider	Provides actual installed/available state from package manifests, asmdefs, provider registration, define symbols, or reflection-safe detection.
PungentPackagePresenceResolver	Combines manifest, asmdef, provider, bridge, and package catalog metadata into a stable snapshot.
PungentPackagePresenceSnapshot	Immutable view of installed, missing, disabled, simulated, version mismatch, and unsupported package states.
PungentPackageSimulationOverrides	Developer Mode only. Allows testing missing package states and records that the state is simulated.
PungentPackageGateCard	Common UI card/chip showing owner, missing state, disabled action, install hint, and safe fallback actions.

Missing package behavior

- Keep discoverable records visible where discovery is useful.
- Preserve metadata, package owner, stable ID, tags, backlinks, summary, and raw asset/file references.
- Show a fallback preview where available.
- Disable unsupported actions with a clear reason.
- Offer safe actions: Copy ID, Copy path/link, Ping asset/path, Open raw asset/file, Open documentation, Create linked note when safe.
- Show which package or bridge provides the missing capability.
- Do not remove, corrupt, rewrite, or silently hide data because a concrete package is absent.

Package simulation rules

- Simulation controls require Developer Mode.
- Simulated states must be labeled as simulated in package cards, utility cards, and validation logs.
- Simulation must not change source package manifests or asmdefs.
- Simulation must not bypass real compile-time dependency constraints.
- Validation should include Core-only, Core plus one extension, full bundle, missing optional bridge, and simulated missing package states.

9. Shared Authoring Data Foundation Doctrine

The Authoring Data Foundation is a Core-owned shared model/contracts layer. It allows notes, rich documents, board documents, data sheets, tasks, help topics, token definitions, documentation links, audit issues, utilities, and external targets to share IDs, references, metadata, preview actions, open/copy/ping behavior, and validation results without forcing all editor packages to depend on each other.

Browser versus editor surface separation

Surface	Owns	Does not own
Authoring Browser	Browse, filter, search, metadata, backlinks, context links, read-only previews, provider status, package state, launch actions.	Full editing of rich documents, boards, sheets, or package-specific workflows.
Rich Document Editor	Text authoring, document structure, inline token insertion, templates, document-specific commands.	Global authoring registry, board editing, table editing, or package presence management.
Board / Whiteboard Editor	Spatial cards, nodes, relationships, dependency maps, canvases, graph minimaps.	Long-form prose editing or CSV/table workflows.
Spreadsheet / Data Sheet Editor	Tables, CSV, scanner-generated sheets, token/content/data coverage, structured data editing.	Rich prose editing or graph layout.
Shared Authoring Foundation	Stable IDs, link targets, metadata, preview/action/validation contracts, migration fields, provider registration, missing-provider fallback.	Concrete editor UI or domain-specific workflows.

Core-owned model scope

- Stable authoring ID helper.
- Authoring item kind enum or registry: LegacyNote, RichDocument, BoardDocument, DataSheet, Task, HelpTopic, TokenDefinition, DocumentationLink, AuditIssue, Utility, ExternalTarget.
- Authoring link target model: asset GUID, local file path, web URL, utility ID, future utility ID, help topic ID, token key, note/document ID, board ID, sheet ID, audit issue code, script path, component type, serialized property path, scene object global ID, and external path.
- Shared metadata: title, summary, tags, status, priority, visibility, created/updated timestamps, source, package owner, extension owner, stable key, archived flag, developer-only flag, and migration version.
- Shared open/copy/ping/action contract.
- Shared preview provider contract.
- Shared validation result contract.
- Shared migration/version fields.
- Optional provider registration for document, board, sheet, note, token, help, documentation-link, audit, utility, and external target providers.
- Safe missing-provider behavior.

Provider contracts

- IPungentAuthoringItemProvider
- IPungentAuthoringPreviewProvider
- IPungentAuthoringActionProvider
- IPungentAuthoringValidationProvider
- IPungentAuthoringSearchProvider
- IPungentAuthoringMigrationProvider

Legacy Notes and Roadmap migration doctrine

Do not remove the Notes system. Preserve the robust browser, automation, metadata, context menu, inspector, scene, audit, token, help, documentation-link, and utility integration features. Deprecate only the old freeform body editing panel inside Notes and Roadmap.

- Keep legacy notes browseable and manageable records.
- Expose PungentNote records through a LegacyNote authoring provider.
- Map existing note targets into shared authoring link targets.
- Show the legacy body as read-only preview/summary in the Authoring Browser.
- Route real editing to Rich Document Editor when installed.
- When Rich Document Editor is absent, show read-only preview plus copy/export/missing package actions.
- Preserve stable IDs, backlinks, context menus, inspector summaries, scene badges, help bridges, audit bridges, token links, and documentation-link relationships.
- Migrate legacy note bodies by linked copy first. Do not destructively convert or delete PungentNote.body.

Developer metadata gate for authoring records

Archived, developer-only, hidden, generated, internal, source, or package metadata controls in authoring records require the Developer Mode metadata mutation gate. When disabled, records may show badges but must not expose mutable developer metadata controls.

Concurrency rule for documents, boards, and sheets

Rich Document Editor, Board/Whiteboard Editor, and Data Sheet Editor may proceed concurrently only after the Authoring Data Foundation contracts compile and the Legacy Note provider demonstrates the shared ID/link/metadata/preview/action model. No track may invent its own permanent ID, link, metadata, action, or validation model unless it first proves the shared foundation cannot represent the need.

10. Source-Truth and Verifiability Doctrine

This section prevents cleanup summaries from being treated as implementation evidence. A debrief is useful context, but source reality and active UI reality win.

Current files outrank all debriefs

A source archive or repository checkout is the source of truth for implementation state. Debriefs, historical briefs, and conversation summaries describe intent, plan, or claimed outcome. They do not prove active implementation. Agents and reviewers must distinguish stated intent, claimed outcome, source reality, and active UI reality.

Evidence levels

Evidence	Acceptable for	Not sufficient for
User current instruction	Intent, priority, override.	Claiming implementation completed.
Current source file	Source implementation.	UI visibility or usability.
Active UI screenshot/manual Unity test	UI reachability, layout, focus, and interaction evidence.	Code architecture correctness alone.
Snapshot audit	Package state at audit time.	Current state after newer archive.
Debrief or summary	Rationale and claimed goals.	Implementation verification.
Historical chat	Context and motivation.	Current status.

No invisible implementation rule

A feature is not functionally implemented if it only exists in code. UI-facing utility work requires a current source path, active call path, registered access surface where relevant, visible and reachable UI location, usable controls at representative sizes, validation of the intended interaction, and no contradictory duplicate stale version elsewhere.

Claim evidence format

- Feature/control.
- Previous location, if known.
- Current source owner: file, class, method, and state owner.
- Current active UI path: menu, window, tab, panel, foldout, overlay, tray, or visibility condition.
- Reachability condition.
- Developer Mode/package/selection prerequisites.
- Validation performed.
- Known caveat.

Active call path requirement

9. Menu, launcher, context menu, browser row, tray, inspector, overlay, or shortcut opens the expected surface.
10. Window state selects the expected tab/panel/module.
11. The active draw/create method includes the expected section.
12. The section uses current data/state, not stale duplicates.
13. The control is visible and interactable under normal conditions.
14. The action produces or changes the expected state.
15. Developer Mode and package presence gates are honored before drawing and before executing.

11. UI Preservation and No-Hiding Doctrine

UI refactors must improve organization without removing the user ability to perform the workflow. Hiding is not simplifying.

The no-hiding rule

Do not solve a UI issue by hiding functionality users still need. A feature may be collapsed, moved, filtered, or placed behind Developer Mode only when the new access path is intentional, documented, discoverable, and appropriate to its audience.

A hidden feature is a regression if

- The user previously relied on it.
- It remains part of intended product scope.
- No replacement path exists.
- It moved to a less discoverable location without redirect or handoff.
- It is only reachable through an obscure toggle.
- It is technically present but visually clipped, cramped, or below unusable scroll depth.
- It appears in multiple places and the active one is unclear.
- It requires Developer Mode but is actually a normal user workflow.

UI relocation contract

- Before map: list each existing visible control/section and where it lives.
- After map: list the new location and access path.
- Rationale: explain why the new location better matches user workflow.
- Redirect: provide related-tool link, inline handoff, compatibility alias, or clear label if the old place no longer hosts it.
- Visibility: ensure the new location is visible at practical window sizes.
- Persistence: preserve relevant preferences, splitter widths, selected tab, search text, filters, tray state, and foldouts.
- Validation: test the moved control after compile/domain reload.

UI control inventory gate

Before any cleanup pass on a window, create a control inventory covering entry point, tabs/modules/foldouts, visible sections, actionable controls, filters/search controls, dangerous actions, diagnostic/status panels, developer-only controls, documentation/help links, overlays/trays, package states, and optional integration states. After the pass, every control must be preserved, moved with path, replaced by equivalent, intentionally removed with approved rationale, or gated behind a documented mode.

Duplicate and stale UI rule

Duplicated UI can be worse than missing UI. If two panels appear to expose the same feature but only one is active or current, remove or clearly mark stale panels, keep compatibility aliases only at access-surface level, route old entries to the new active implementation, avoid identical labels for different behavior, and include a deprecation warning only when the old path still exists.

Cramped UI is not completed UI

A feature does not count as restored if controls exist but are squeezed into unreadable columns, clipped labels, tiny hit targets, or scroll traps. Validate narrow width, comfortable width, wide width, tall/short heights, docked/floating mode, foldout/tab states, empty/partial/populated data states, and missing package states.

12. Editor UI and UX Principles

The suite should have a cohesive visual identity, but not a rigid one. Consistency comes from shared theme colors, typography rules, spacing scale, button/status/box styles, icons, interaction conventions, safety patterns, registry metadata, lab navigation, common window chrome, and reusable widgets. Consistency should not mean every workflow is forced into the same vertical inspector layout.

Unity-native alignment

- Buttons trigger actions.
- Toggles enable states.
- Dropdowns choose one option.
- Radio groups choose mutually exclusive modes.
- Sliders adjust continuous ranges.
- Numeric fields set exact values.
- Object fields reference assets or objects.
- Search fields filter.
- Tabs separate workflows.
- Toolbars collect compact actions.
- Progress bars report long work.
- Help boxes explain state.

Responsive layout rules

- The window must remain usable at small sizes.
- Prefer vertical scrolling over horizontal scrolling.
- Use wrapping rows, compact cards, collapsible sections, tabs, paging, and column stacking before horizontal scrolling.
- Use draggable splitters where panel proportions matter, and persist sizes.
- Avoid dead space, jumpy controls, and important actions buried below large scroll regions.
- Responsive means key controls remain discoverable and practical, not merely that the UI technically draws.

Visual information design

- Use `UtilityWindowTheme` for shared ImGui chrome.
- Use equivalent design tokens for UI Toolkit tools.
- Use color intentionally for status, category, severity, selection, lab identity, package state, or risk.
- Do not use color as the only differentiator.
- Use icons, tags, badges, count chips, legends, labels, and tooltips for repeated states.
- Use diagrams, mini graphs, swatch grids, matrices, timelines, heatmaps, and graph canvases only where they clarify faster than text.

Decision-density doctrine

Common workflows should not ask the user to make too many decisions at once. Show the primary workflow, summarize state, expose advanced settings contextually, and keep rare or risky actions visually separated. Advanced options can be collapsed only when the core workflow remains complete.

13. Window and Panel Layout Doctrine

Each window and panel should be designed deliberately. A utility layout is part of the product design, not decoration.

Layout templates

Template	Best for	Doctrine
Simple vertical stack	Small settings and single-purpose helpers.	Header -> configuration -> action -> status/help. Avoid for dense or comparison-heavy workflows.
Two-column layout	Source/result, configuration/preview, selection/detail.	Use persistent draggable splitters where both sides compete for space.
Three-panel layout	Advanced authoring, asset browsers, debug tools, lab hubs.	Left navigation/source, centre workspace/results, right details/actions. Collapse or stack at narrow sizes.
Grid/card layout	Asset sets, palettes, icons, prefab previews, generated names, tool launchers.	Cards should wrap and use compact metadata.
Table/matrix layout	Coverage tools, validation, dependency maps, compatibility checks.	Use sorting, filtering, grouping, and detail panes. Horizontal scroll is acceptable when preserving column relationships matters.
Canvas/lab workspace	Generation, placement, spatial workflows, graph-like relationships.	Central workspace, side configuration, contextual toolbar, mode-specific controls.
Wizard/step flow	Setup, migration, export, risky multi-stage operations.	Show step, inputs, validation, dry-run results, and final apply/export.
Dashboard/overview	Control panel, lab home, health summaries.	Surface summaries, warnings, search, recent tools, quick actions. Do not turn dashboards into dense control panels.
Contextual mini panel	Component, scene selection, terrain, asset quick actions.	Keep compact and include an Open Full Tool handoff for complex work.

Shared resizable panel contract

- Handle orientation matches motion.
- Drag direction is intuitive.
- Handle placement matches ownership.
- Panel edges stick to handles.
- The remaining panel absorbs remaining space.
- Bounds are relative, not arbitrary hard-coded maxima.
- Saved splitter values are clamped before drawing.
- Handles remain discoverable and have comfortable hit targets.
- Handle thickness and spacing are included in layout calculations.
- Every touched splitter is validated by dragging in both directions.

Canonical split calculation rule

16. Reserve fixed outer padding, inner spacing, and handle width/height.
17. Compute minimum sizes for each neighboring panel.
18. Clamp the saved size against current available area and sibling minimums.
19. Draw the fixed or user-sized panel at the clamped size.
20. Draw the handle immediately adjacent to the panel it controls.
21. Draw the remaining panel with expansion so it consumes leftover space.

Adaptive row wrapping contract

- Controls that form one conceptual row should remain in the same row when there is room.
- Separate semantic rows should stay separate.
- Rows should wrap by whole control groups, not by splitting labels away from fields or icons away from labels.
- Each control group should have minimum, preferred, and maximum widths.
- When a row wraps, preserve reading order and keep primary actions discoverable.
- Horizontal scrollbars are a last resort for normal controls and acceptable only for inherently horizontal artifacts.

Layout anti-patterns

- Long undifferentiated walls of fields.
- Excessive nested foldouts.
- Large empty gaps from rigid sizing.
- Avoidable horizontal scrollbars.
- Controls that jump during refresh.

- Important actions hidden below large scroll regions.
- Duplicate buttons doing the same thing.
- Making every utility look identical at the cost of usability.
- Using graph/canvas/wizard UI only because it looks advanced.
- Claiming a section was restored when it exists only in code but not in the active UI.

14. Overlay Tray and Popup Surface Doctrine

Overlay trays should be the preferred surface for transient popup-style interactions when the task is contextual, lightweight, dismissible, and related to a currently visible utility, browser, panel, tray, or selection. Prefer a shared overlay tray over an ad hoc detached popup EditorWindow, UtilityWindow, floating mini-window, or modal dialog whenever the user benefits from staying in the current workflow.

Use overlay trays for

- Contextual help, glossary, and documentation previews that need an Open Full Help Browser handoff.
- Minimized utility controls, recent tools, quick actions, and utility switchers.
- Lightweight settings, filters, sort controls, package state explanations, and missing dependency notices.
- Selected-item summaries, backlinks, related utility lists, authoring previews, and bridge/action menus.
- Small pickers, inspectors, confirmation previews, and temporary palettes that support an active panel.
- Developer Mode gate explanations when a compact surface needs to explain hidden controls without exposing them.

Use a full EditorWindow instead when

- The workflow needs a persistent dockable workspace.
- The user needs large result review, dense tables, graph/canvas editing, rich document editing, sheet editing, scene authoring, or multi-stage scan/apply flows.
- The surface needs long-lived authoring state or complex keyboard/navigation behavior.

Use a modal dialog only for

- Destructive confirmation.
- Blocking error recovery.
- Required save/discard decisions.
- Security/safety choices where the user must answer before continuing.

Overlay tray manager contract

Contract element	Purpose
ownerUtilityId	Identifies the utility or package that owns the tray.
ownerSurface	Window, row, toolbar, overlay, tray, inspector, selection, Scene View, or browser card.
topicId / contextId	Help, documentation, selected item, package, scan, or action context.
preferredPlacement	Right rail, bottom rail, anchored popup, inline tray, or floating tray.
canPin	Whether outside-click dismissal is allowed.
fullToolHandoff	Destination when content outgrows the tray.
developerGate	Optional Developer Mode gate.
missingPackageState	Optional dependency fallback state.

Overlay tray behavior rules

- A tray must have a clear owner: utility window, panel, toolbar button, selection, scene overlay, minimizer strip, or browser row.
- A tray should be non-modal by default, easy to dismiss, and should never trap the user unless the task is intentionally blocking.
- A tray must clamp within usable editor/window bounds, handle narrow and short sizes, and avoid covering the primary action that opened it unless no better placement exists.
- User-movable or resizable trays must persist and safely clamp position, size, collapsed state, and selected topic after domain reload, docking changes, and monitor changes.
- Trays should use shared theme tokens, calm borders, readable elevation/shadow, clear title/action rows, and visible focus states.
- Escape should close dismissible trays. Clicking outside may close lightweight trays unless pinned.
- Trays that contain more than quick content must include Open Full Tool, Open Full Browser, or Dock as Window handoffs.

- A tray must render cached state and invoke explicit actions; it must not own project scans, heavy reflection, asset mutation, scene mutation, or broad document generation from repaint.
- Multiple trays should be coordinated by a shared tray manager or rail. Avoid overlapping floating-popup clutter, duplicate tray stacks, and hidden tray state with no visible affordance.

Overlay tray implementation review gate

- State why the workflow is tray-appropriate rather than a full EditorWindow, inspector, scene overlay, context menu, or modal dialog.
- Identify the owner surface and active UI path.
- Verify narrow/wide, short/tall, docked/floating, pinned/unpinned, outside-click, Escape, domain reload, focus, and missing-provider states.
- Confirm the tray does not hide essential controls, does not break keyboard focus, and provides a full-tool handoff when content outgrows the tray.

15. Progressive Disclosure, Assistance, Accessibility, and Messaging

Progressive disclosure rules

- Show the core workflow first.
- Collapse advanced options by default only if the core workflow remains complete.
- Do not bury essential controls under advanced sections.
- Do not move required controls into Developer Mode unless they are truly developer-only.
- If a feature is filtered out, provide visible filter state and a way to reveal it.
- If a feature is archived or hidden, provide restore or show-hidden controls where appropriate.

Tooltips and contextual help

- Every meaningful field should have a tooltip or help affordance.
- Tooltips explain what the field controls, when to change it, how it differs from related options, and whether it is destructive, experimental, expensive, developer-only, or preview-only.
- Complex enums should have nearby descriptions.
- Labels should describe the enabled state or action clearly.
- Help affordances should preserve context: utility ID, package ID, tab/module, section, selected item, and topic anchor.

Accessibility expectations

- Do not rely on color alone.
- Pair color with icons, labels, shapes, patterns, or tooltips.
- Maintain readable contrast.
- Provide visible focus states where feasible.
- Preserve logical keyboard navigation where supported.
- Avoid tiny click targets for frequent actions.
- Use plain-language labels for common operations.
- Runtime UI and Feedback Lab components should expose meaningful accessibility hierarchy, labels, roles, values, and state where supported.

Empty states and errors

Empty states should explain what is missing and offer one clear next action. They should not use error styling unless something failed. Errors state the problem, likely cause, and next useful step. Warnings identify risk before changes are applied. Info messages confirm state without blocking workflow. Missing package states are not errors unless a required dependency is unexpectedly unavailable.

16. Unity Editor Implementation Doctrine

PungentFunk tools should choose Unity Editor implementation patterns deliberately. This is a selection framework, not a mandate to rewrite stable tools.

IMGUI, UI Toolkit, and migration policy

- IMGUI remains acceptable for existing windows, small utility panels, debug tools, compatibility wrappers, and internal workflows where code-driven rendering is fastest and clearest.
- UI Toolkit should be evaluated first for new flagship lab windows, persistent complex workspaces, product-facing tools, rich styling, component-rich views, retained state, UXML/USS-driven presentation, and tools expected to grow significantly.

- Do not rewrite for fashion. Rewrite when there is a concrete product, maintainability, accessibility, performance, or UX reason.
- Hybrid strategies are acceptable.
- Major tools should document why they use IMGUI, UI Toolkit, CustomEditor, Overlay, TreeView, graph/canvas, or a hybrid pattern.
- Both IMGUI and UI Toolkit tools must consume the same shared contracts for Developer Mode, package state, help context, minimize/tray, and window chrome where applicable.

Implementation role selection

Role	Use when	Doctrine
EditorWindow / Lab Window	Workflow has filters, previews, generated results, scan/apply stages, dashboards, tabs, or persistent workspace.	Keep scan/apply logic outside repaint. Cache expensive state. Use shared chrome.
CustomEditor	Tool belongs directly to a selected component or asset.	Keep compact. Provide validation, setup helpers, package messages, and Open in Lab handoffs.
PropertyDrawer	A serializable field type needs reusable presentation.	Use for field-level UX, not whole workflows.
Scene GUI / Handles	User edits spatial data directly in Scene View.	Keep lightweight. Respect Undo, snapping, visibility, selection, and Prefab Mode.
Scene View Overlay / EditorTool	Persistent compact controls are needed while working in Scene View.	Use for placement, navigation, debug toggles, paths, terrain/surface, and environment previews.
TreeView / MultiColumnView	Data is hierarchical, dense, expandable, or audit-like.	Preserve row identity, selection, expansion, sorting, filtering, and scroll position.
Search Provider / Query Tool	Dataset is large and users think in terms of discovery/filtering.	Avoid inventing custom query syntax unless the lab needs it.
Shortcut / Command	Action is frequent, safe, contextual, and easy to describe.	Avoid rare or destructive shortcuts. Developer actions must remain gated.
Context Menu	Short action is most useful from selection.	Keep concise and route full workflows to labs.
Graph / Canvas Tool	Relationships, branching, dependencies, or pipelines are the primary authoring problem.	Use canvas only where it clarifies more than forms, lists, or tables.
Optional Bridge	Feature links labs or optional packages.	Keep removable, isolated, and gracefully degraded.

17. Safety and Performance Doctrine

Drawing must not own mutation

Editor drawing methods should render current state. They must not own expensive scans, asset writes, scene mutation, graph mutation, broad reflection work, documentation generation, script parsing, or project-wide operations.

Safe broad operation flow

- 22. Configure.
- 23. Scan or validate.
- 24. Preview.
- 25. Apply selected or apply all.
- 26. Summarize changed, skipped, and failed items.

Performance standards

- Do not scan all scenes or assets on every visual refresh.
- Cache scan results and rebuild only when settings change or the user refreshes.
- Use staged editor work for long preview, export, bake, icon, or scan operations.
- Use cancelable progress for expensive work where practical.
- Avoid Thread.Sleep in editor tools.
- Avoid broad SceneView repaint unless required.
- Preserve row/node identity in large lists, trees, tables, and graphs.
- For UI Toolkit, use virtualization/pooling where large repeated lists are involved.
- Static state is a cache, not source-of-truth user data.

Safety standards

- Preview or dry-run before destructive and batch operations.
- Default dangerous toggles to safe behavior.
- Do not overwrite user data without explicit consent.
- Do not modify shared materials, prefab assets, imported assets, or scenes by default without warning.
- Show a summary of changed, ignored, and failed items.
- Be explicit about scope: selection, active scene, open scenes, prefab assets, project assets, generated groups, or all matching files.
- Use Undo for scene/object changes and SerializedObject/SerializedProperty where prefab overrides, multi-object editing, or Undo matters.

18. Shared Package Systems and Project Scan Pipeline Doctrine

Project-wide scanning is a shared package service, not a per-window implementation detail. Future panels that need broad project facts should consume shared scan contracts, provider registry, cache, project audit index, and shared findings UI before creating local scanners.

Shared systems first rule

Before adding a local registry, scanner, cache, documentation index, metadata store, status vocabulary, progress model, or result UI, check whether Core already owns or should own a shared contract. If the result could be reused by another tool, register a provider or shape the local result so it can be promoted later.

Scan ownership model

Owner	Owns
Core	Scan contracts, result models, severity vocabulary, provider registration, project audit cache/index concepts, cooperative scan coordination, progress/state presentation helpers, and background/idle scheduling policy.
Extension packages	Domain-specific scan providers, advanced scan UIs, domain fixes/actions, and package-specific interpretation of findings.
Bridges	Cross-package scan exports, optional sheet/document/board/finding handoffs, third-party scan adapters.
Project adapters	Game-specific validators and adapters outside generic packages.

When to register a scan provider

- The scan produces reusable project facts, findings, coverage, dependencies, token data, documentation state, package metadata, or release readiness state.
- Another utility could reasonably consume the results.
- The scan may become expensive enough to need scheduling, cancellation, stale-state reporting, or caching.
- The scan should appear in package-wide summaries or findings UI.

Shared scan result contract

- Provider ID and package owner.
- Scan scope and source.
- Timestamp, duration, cache freshness, stale state.
- Severity, category, title, summary, detail, target references.
- Suggested actions, fix availability, dry-run support, and validation links.
- Stable finding IDs where possible.
- Optional sheet/export/authoring link metadata.

Background and immediate scan doctrine

Background scans should be cooperative, idle-aware, pause/cancel aware, and unobtrusive. Immediate scans may be faster and more intrusive, but should be explicitly user-triggered and should still report progress, cancellation, and stale cache state where possible.

Cache and stale-state doctrine

Scan-consuming windows should show cached results on open, communicate last scan age and stale state, avoid forced rescans unless explicitly requested or safely scheduled, and provide refresh controls that state their scope and expected cost.

Scan UI doctrine

Findings UI must be readable, grouped, filterable, and action-oriented. Dense findings cards should surface severity, title, target, package/provider, freshness, and the most useful action first. Details, raw identifiers, and developer-only diagnostics should be progressively disclosed.

Repaint safety for shared services

No scan, indexing, reflection, script parsing, documentation generation, or broad project query should run from routine repaint. Drawing code draws cached state. Services produce state through explicit actions, staged editor work, background tasks, or controlled provider refreshes.

19. Editor Access and Integration Principles

Menu items are the baseline access method, not the only access method. Tools should appear where the user naturally needs them.

Access method	Principle
Organized menus	Use clean, curated menu roots. Avoid duplicate clutter and old project-specific names except as compatibility aliases.
Utility Browser / Lab Hub	Every significant utility should register with the central registry and be searchable/filterable.
Toolbar / command access	Use sparingly for frequent global or status-like actions. Do not mirror the full menu tree.
Scene View overlays	Use for placement, surface alignment, navigation, gizmo browsing, debug visual toggles, path tools, terrain/surface helpers, and environment previews.
Context menus	Use for concise selected-object, asset, folder, material, prefab, texture, terrain, palette, or ScriptableObject actions.
Inspector integration	Use compact toolbars, validation panels, quick actions, dependency messages, and Open in Lab buttons.
Cross-utility embedding	Use optional services, registry lookup, package detection, reflection-safe adapters, or bridge packages.
Shortcuts	Use for frequent, safe, easy-to-describe commands; developer actions remain gated.
Context-aware launch	Preload relevant selection context and fall back gracefully.
Recent/pinned/resume	Preserve recent tools, favorite labs, last active module, tray state, and related-tool handoffs through shared preferences.

Access-surface preservation

When moving or consolidating access paths, preserve discoverability. Legacy aliases may route to the new active tool but should not maintain competing stale implementations. If a utility becomes package-gated, the access surface should explain which package owns it and what functionality remains available.

Menu policy

- Primary editor menus: Tools/PungentFunk Utilities/...
- Asset context menus: Assets/PungentFunk Utilities/...
- GameObject context menus: GameObject/PungentFunk Utilities/...
- CreateAssetMenu root: PungentFunk Utilities/[Lab]/[Asset Type].
- Legacy aliases are allowed only as compatibility routes and should be marked internally.
- Package-specific menu entries belong to extension packages or bridge packages, not Core menu owners.

20. Optional Dependency and Cross-Utility Integration Principles

PungentFunk Utilities should become more powerful when multiple labs are installed together without making standalone labs fragile.

Dependency levels

Level	Meaning
Required dependency	Must exist for the package to compile and function. Keep minimal and documented.
Optional integration	Activates when another lab or package is installed. Must not compile-break when absent.
Suggested companion	Improves workflow but is not required. Missing companion messaging should be calm and non-blocking.
Bridge dependency	Isolated package or asmdef that can reference both sides and be removed without breaking either side.

Optional integration techniques

- Assembly definition optional references.
- Version defines.
- Define constraints.
- Reflection-safe discovery.
- Interface packages.
- Adapter classes in separate integration folders.
- Registry-based service lookup.
- Loose ScriptableObject references.
- Conditional compilation.
- Soft dependency metadata in descriptors.
- Disabled-but-helpful UI state when absent.

Cross-lab dependency rule

Cross-lab dependencies are product decisions, not accidental using directives. If a feature links two labs, prefer a bridge. Direct lab-to-lab compile dependencies are acceptable only when the dependency is required and documented for that product package.

21. Runtime, Editor, Data, and Serialization Rules

Runtime package rules

- Runtime scripts must compile in player builds without UnityEditor.
- Runtime data models need stable serialized fields, version fields, and migration-safe defaults.
- Avoid editor-only assumptions such as AssetDatabase, EditorPrefs, EditorWindow, GUIContent, or menu paths.
- Public runtime APIs should be conservative and generic.

Editor package rules

- Editor windows should use SerializedObject and SerializedProperty when editing serialized assets/components where Undo, prefab overrides, or multi-object editing matters.
- Use Undo for scene/object changes.
- Use AssetDatabase and EditorUtility.SetDirty safely.
- Prompt before opening/changing scenes, editing prefab assets, or applying destructive project-wide changes.
- Separate rendering from scanning, asset mutation, scene mutation, graph mutation, and service logic.
- Editor-only source lives in Editor folders or editor asmdefs.

Serialization and prefab doctrine

Prefab-facing tools must distinguish prefab assets, prefab instances, variants, nested prefabs, Prefab Mode isolation, and in-context prefab editing. Do not silently apply scene-instance choices to prefab assets. Generated assets should preserve GUID/meta stability where possible and avoid delete/recreate churn when update-in-place is safe.

Authoring data serialization doctrine

Authoring item IDs and link targets must be stable across package installs, domain reloads, asset moves, optional package availability changes, and missing providers. Missing providers should not delete or rewrite persisted authoring records. Migration/version fields should be explicit and forward-compatible.

22. Asset, Package, and Import Pipeline Doctrine

Package and distribution doctrine

- Design labs so they can ship as standalone products and as a bundle.
- Use package boundaries, asmdefs, samples, documentation, icons, presets, and optional bridges deliberately.
- Keep content organized: runtime code, editor code, samples, documentation, icons, presets, migration helpers.
- Do not rely on project-specific folder paths.
- Default output folders should be configurable, safe, and visible.
- Use special Unity folders deliberately and sparingly.

AssetDatabase and import workflow doctrine

- AssetDatabase is editor-only.
- Asset operations should be explicit and previewed when they affect many assets.
- Batch operations should avoid unnecessary refresh/import churn.
- Tools should distinguish source assets, generated assets, temporary previews, and user-authored assets.
- Generated asset names should be deterministic where useful, collision-safe, and reversible.
- Generated docs must not overwrite curated docs unless the user explicitly requests replacement.

23. Package Split Map

This package map defines recommended ownership for release planning and implementation passes until superseded by a formal package manifest. The current source may still be monolithic in places; this map is the target boundary doctrine.

Package	Package ID	Tier	Provides	Release rule
Free / Core	com.pungentfunk.utilities.core	Core	Registry/descriptors, Utilities Browser, theme/preferences, shared chrome, minimizer/tray, help read/open, package presence, Developer Mode gates, shared scan contracts/cache shell, token parser contracts, Authoring Data Foundation contracts, lightweight Authoring Browser shell.	Must not compile against concrete lab windows or own advanced editors.
Full Bundle	com.pungentfunk.utilities.full-bundle	Bundle	All extensions, all official bridges, samples, bundle landing pages, coordinated dashboards.	Convenience only; not the only working configuration.
Documentation and Authoring	com.pungentfunk.utilities.authoring	Extension	Rich Document Editor, advanced Authoring Browser, legacy Notes manager, help authoring, documentation-link editing, templates, token insertion helpers.	Serious writing/editing belongs here; reduced browsing shell can remain Core.
Board / Graph / Visualization	com.pungentfunk.utilities.board	Extension	Whiteboard/board documents, node graph editor, relationship maps, graph minimap, blackboard data.	Use shared authoring IDs/links; no separate permanent link system.
Spreadsheet / Data Sheet	com.pungentfunk.utilities.datasheet	Extension	Data Sheet Editor, table model, CSV import/export, scanner-generated sheets, token/content sheets, Coverage Matrix successor.	Sheet editor works without Audit; scanner sheet actions bridge later.
Audit / Validation / Scanning	com.pungentfunk.utilities.audit	Extension	Design Validation Audit advanced UI, project audit providers, reference scanner, terrain scanner, token validator advanced scan UI, findings actions, release	Core owns scan contracts only; domain providers live here or in domain packages.

Package	Package ID	Tier	Provides	Release rule
			readiness.	
Scene Authoring / Placement	com.pungentfunk.utilities.scene-authoring	Extension	Asset Placement Lab, Surface Align, Modular Path Builder, Scene Navigation, Area Authoring Toolkit, path/placement repair, spatial actions.	Strong candidate for early extraction with own runtime/editor asmdefs.
Scene Gizmos	com.pungentfunk.utilities.scene-gizmos	Extension	Scene Gizmo Browser, gizmo source/editor, beacon/sensor/trajectory visualizer, provider cache, performance triage.	Can be separate or folded into Scene Authoring by manifest decision.
Asset Production	com.pungentfunk.utilities.asset-production	Extension	Prefab Icon Generator, Prefab Asset Exporter, Font Preview, Texture Array Baker, preview/export queues.	Mostly editor-only; optional bridges to Appearance and Audit.
Appearance / Colour / Texture	com.pungentfunk.utilities.appearance	Extension	Palette Designer, colour analysis/accessibility, theme preview integrations, procedural texture workflows.	Core keeps theme contracts; palette/texture workflows live here.
Audio Authoring	com.pungentfunk.utilities.audio	Extension	Audio Setup Coverage, Audio Catalog Coverage, audio clip/profile/matrix authoring, terrain/material audio profiles, audio validation providers.	Requires runtime/editor split.
Debug / Diagnostics	com.pungentfunk.utilities.debug	Extension or developer-focused	Debug Controller, DebugRouter panels, scheduled actions, signal/channel diagnostics, reflected component state panels.	Developer-only controls still require Developer Mode.
UI / Input Feedback	com.pungentfunk.utilities.ui-input	Extension	Input Prompt Icon Library, input prompt/token bridges, UI feedback presenters.	Potential free runtime subset later.
Content Generation	com.pungentfunk.utilities.content-generation	Extension	Name Generator, name-list tools, MadLib tools, future text generation helpers, document/content export bridge.	Good lightweight standalone extension.
Project adapters	varies	ProjectAdapter	Game-specific presets, compatibility aliases, game data translators.	Outside generic packages.

Package validation matrix

Configuration	Validation expectation
Core only	Compiles with no lab windows. Utilities Browser opens. Theme, tray, help browse, package cards, Developer Mode root gate, scan contracts, authoring contracts, and missing-extension cards are usable.
Core + Documentation and Authoring	Authoring Browser opens, legacy notes browse correctly, rich document editor opens if installed, old note body editing is not the primary workflow.
Core + Scene Authoring	Placement/path/gizmo/navigation tools compile and register without Audit, Audio, Colour, or Asset Production installed.
Core + Audit	Advanced audit UI opens using Core scan contracts. Missing domain providers are shown as absent, not broken.
Core + Data Sheet	Sheet editor and CSV support compile without Audit; scanner-generated sheet actions are disabled until Audit bridge exists.
Core + Board	Board editor compiles and previews generic authoring references without Documentation and Authoring installed.
Full Bundle	All utilities and bridges work, but tests must not rely only on this configuration.

24. Lab Taxonomy

Lab	Product role	Primary ownership
Utility Core	Shared shell and UX infrastructure.	Registry, browser, menus, tray/minimizer, theme/prefs, shared UI contracts, Developer Mode, package presence, scan/result models, Authoring Data Foundation contracts.
Documentation and Authoring	Focused rich document and documentation authoring.	Rich Document Editor, advanced Authoring Browser, help/docs authoring tools, templates, legacy notes migration.
Board / Graph / Visualization	Spatial and relationship-heavy authoring.	Board documents, node graphs, dependency maps, relationship canvases, graph previews.
Spreadsheet / Data Sheet	Structured tabular authoring and coverage data.	Data sheet editor, CSV import/export, scanner sheets, Coverage Matrix successor.
Audit / Validation / Scanning	Project health, validation, release readiness, coverage.	Design Validation Audit advanced UI, providers, findings actions, advanced token scan UI.
Scene Authoring / Placement	Reusable scene authoring workflows.	Placement, alignment, paths, area authoring, scene navigation, scene gizmos if grouped.
Asset Production	Preview, generation, export, and production utilities.	Prefab icons, prefab export, font preview, texture arrays, preview/export queues.
Appearance / Colour / Texture	Colour, palette, theme, visual analysis, procedural appearance.	Palette Designer, colour accessibility, theme preview, procedural texture generation.
Audio Authoring	Audio setup, catalog, coverage, and validation.	Audio coverage windows, audio profiles/matrices, audio validators.
Debug / Diagnostics	Debug control and diagnostic observation.	Debug Controller, routers, scheduled actions, signal/channel diagnostics.
UI / Input Feedback	Prompt/icon libraries and UI feedback surfaces.	Input prompt icon library, input token bridges, feedback presenters.
Content Generation	Lightweight reusable content generation.	Name Generator, name lists, MadLib assets, content export bridges.

25. Documentation Architecture

The design bible should not carry every task. Documentation should be layered so humans and AI agents can find the correct source quickly.

Layer	Artifact	Purpose	Volatility
Vision	docs/design-bible/index.md or vision.md	Short alignment, product promise, labs, reading paths.	Low
Doctrine	docs/design-bible/doctrine.md and tagged PDF editions	Stable architecture, UX, safety, package doctrine.	Low
Patterns	docs/design-bible/patterns/*.md	Concrete reusable implementation patterns.	Medium
Labs	docs/design-bible/labs/*.md	Per-lab scope, dependencies, workflows, definition of done.	Medium
Agent instructions	.github/copilot-instructions.md, .github/instructions/*.instructions.md, skills/SKILL.md	AI behavior, repo rules, path-specific rules, task procedures.	Medium
Snapshots	Documentation/Snapshots/*	Script counts, feature inventory, audit reports.	High
Execution	Issues, PRs, briefs, update-pass notes	Work planning and implementation deltas.	High
Release	CHANGELOG.md, release notes, tagged PDFs	Human-readable change history.	Medium

Documentation standard

- Every shipped package needs README, setup notes, known limitations, menu paths, package dependencies, example workflows, and package validation matrix.
- Every utility descriptor should link to a help topic or documentation page.
- Every extension should document required and optional dependencies.
- Every bridge should document both sides and missing-package behavior.
- Every migration should document source data, target data, reversibility, and validation.
- Every developer-only feature should document the gate required to expose and execute it.

26. AI Task Routing and Failure Prevention Matrix

AI agents should interpret this bible as a normative decision system, not decorative background. The goal is fewer local one-off UI decisions, fewer hidden regressions, and more consistent implementation outputs.

Agent priorities

- 27. Identify affected package layer: Core, lab, runtime model, editor tool, bridge, project adapter.
- 28. Identify intended implementation role: EditorWindow, CustomEditor, PropertyDrawer, Scene GUI, Overlay, TreeView, Search Provider, Shortcut, Runtime Service, Bridge, Graph Tool, Tray, or hybrid.
- 29. Check whether the existing implementation role matches the user workflow.
- 30. Preserve runtime/editor separation.
- 31. Keep generic packages free of game-specific references.
- 32. Keep optional integrations removable.
- 33. Separate drawing from scanning and mutation.
- 34. Preserve preview/dry-run/apply safety for destructive workflows.
- 35. Use shared Developer Mode gates, shared window chrome, shared tray/minimize contracts, and shared package metadata.
- 36. Update registry/menu/status/documentation metadata when relevant.
- 37. Return concrete validation steps, not vague assurances.

Failure prevention matrix

Risk	Required prevention
Core absorbs extension workflows	Apply Core Anti-Sink Rule; move registrations/actions/providers to package registrars.
Developer controls appear without root toggle	Use Developer Mode gate helpers and test disabled/enabled states across all access surfaces.
Feature exists only in code	Verify active menu/browser/tray/overlay/inspector path and visible usable controls.
Optional dependency compile-breaks	Use bridges, reflection-safe discovery, version defines, or interface contracts. Validate package absent.
Popup clutter	Use shared overlay tray manager for lightweight contextual surfaces.
Window-specific minimize behavior	Route through Core minimizer/tray.
Package state is guessed	Use presence resolver and label developer simulation.
Scan runs during repaint	Use shared scan provider/cache/action flow.
UI refactor hides controls	Create before/after control inventory and validation evidence.
Generated docs overwrite curated docs	Label generated content, preserve curated docs, require explicit replacement.

27. AI Output, Briefing, and Debrief Standards

For audit responses

- Summarize scope and source basis.
- Separate confirmed source facts, active UI facts, inferred risks, and unverified items.
- List affected packages and likely ownership.
- Identify Developer Mode, package, scan, tray, chrome, and source-truth implications.
- Return findings with severity, source owner, active UI path if known, and recommended next action.

For implementation briefs

- State relevant doctrine.
- List target files or discovery instructions.
- Define package ownership and dependency boundaries.
- Define UI/UX requirements, shared chrome/tray requirements, Developer Mode gates, performance requirements, safety requirements, and optional dependency behavior.
- Include validation steps and package matrix tests.
- Use one consistent paste-ready format throughout.

For completed implementation summaries

- Summary of what changed and why.
- Files changed.
- Architecture notes.
- UI/UX notes.
- Developer Mode and package-gate notes.
- Safety/performance notes.
- Validation steps performed.
- Known limitations and explicit unverified items.

Shared scan task-routing rule

Any feature that scans project-wide data, validates package content, indexes tokens/docs/scripts, or reports findings must first route through shared scan contracts or explicitly justify why it remains local. If reusable by another tool, register a provider.

28. Package Hardening Rules

Namespaces

- Use stable generic namespaces rooted in PungentFunk.Utilities or the chosen release namespace.
- Keep labs separated by namespace.
- Keep project adapters outside generic namespaces.
- Avoid names that imply a specific game project unless the code is an adapter.

Assembly definitions

- Runtime and Editor assemblies must be separated.
- Core assemblies must not reference lab assemblies.
- Bridge assemblies may reference both sides and should be removable.
- Use define constraints/version defines for optional third-party integrations.
- Validate compile with optional packages absent.
- Package-specific runtime/editor asmdefs are required before modular release.

Source organization

- Core: contracts, UI helpers, registries, theme, minimizer/tray, help read/open, Developer Mode, package presence, scan contracts, authoring contracts.
- Extensions: concrete windows, scanners, editors, generators, domain models, provider implementations.
- Bridges: optional cross-package handoffs and adapters.
- Project adapters: game-specific translation, aliases, presets, and wrappers.

29. Versioning, Review, and Maintenance

Review cadence

- Update this bible when doctrine changes.
- Update snapshot audits when source changes.
- Update package maps when distribution boundaries change.
- Update lab specs when product workflows or package ownership changes.
- Update pattern pages when repeated implementation patterns emerge.
- Update agent skills/instructions when output format or verification standard changes.

Changelog policy

- Record user-facing changes, package boundary changes, migration behavior, breaking changes, and known limitations.
- Separate doctrine/documentation changes from implementation changes.
- Do not use changelog entries as proof that implementation is active.

Decision log policy

Major package boundary, Core API, scan pipeline, Authoring Data Foundation, migration, distribution, Developer Mode, shared chrome, tray manager, package presence, or UI verification decisions should be recorded in a durable decision log with rationale, alternatives considered, and validation expectations.

30. Utility Definition of Done

A PungentFunk utility update is complete only when the relevant items are satisfied:

- Has a stable registry descriptor and menu/access surface.
- Declares package ownership, package tier, required dependencies, optional dependencies, capabilities, extension points, bridge state, documentation topic, and missing dependency behavior.
- Compiles in intended configurations.
- Preserves runtime/editor separation.
- Does not add Core-to-lab dependency leaks.
- Uses shared theme/layout/state helpers where appropriate.
- Uses shared window chrome or documents a justified exception.
- Uses shared minimize/tray behavior where window minimization is supported.
- Uses shared Developer Mode gates for every developer-only surface or action.
- Uses package presence/fallback UI for optional dependencies.
- Keeps scan/apply/mutation outside repaint.
- Provides preview/dry-run and Undo where needed.
- Handles empty, missing dependency, partial data, stale scan, and error states.
- Is visible, reachable, usable, and verified in active UI.
- Has help/documentation entry or topic link.
- Has concrete validation steps.

31. UI Refactor Definition of Done

For any pass that changes a window, panel, utility browser, debug control center, design validation audit, documentation links window, minimizer, overlay, tray, package card, help surface, or other visible editor UI, apply this stricter gate.

Required artifacts

- Before control inventory.
- After control inventory.
- List of intentionally removed controls.
- List of moved controls with new access paths.
- List of developer-only or hidden controls with rationale and gate.
- Source file/class/method owner for each major section.
- Active draw/create path for each major section.
- Manual Unity validation steps.
- Narrow, comfortable, wide, short-height, tall-height, docked, and floating validation.
- Confirmation that no essential action is only reachable through accidental layout expansion.
- Splitter/handle validation for any touched split layout.
- Adaptive row validation for any touched toolbar/filter/action row.
- Help/context validation for any touched help affordance or documentation link.
- Developer Mode disabled/enabled validation for every touched developer surface.
- Package missing/present validation for every touched package-gated surface.
- Overlay tray validation if trays or popup surfaces are touched.

Refactor acceptance test

A UI refactor is acceptable only when a user can answer these questions without reading source code:

- Where do I start?
- What is selected or filtered?
- What changed after the latest scan/action?
- Where did the controls from the previous version go?

- Which actions are dangerous?
- Which controls are developer-only or hidden?
- How do I restore hidden/archived utilities?
- How do I open the full tool from a compact/contextual surface?
- Why is an action disabled and which package or gate enables it?

32. Consistency Acceptance Grid

Every UI-affecting implementation pass should validate the relevant rows in this grid.

Scenario	Required result
Developer Mode disabled	No developer controls visible or callable. Read-only badges and gate cards are allowed.
Developer Mode enabled	Developer controls appear in predictable labeled locations and remain visibly developer-marked.
Internal source define absent	Source-writing controls do not appear or function.
Core-only package	Utilities Browser, theme, tray, help read/open, registry, scan contracts, package cards, and Authoring shell open; missing editors show disabled helpful states.
Core + one extension	That extension works without full bundle or unrelated extensions.
Full bundle	Bridges and handoffs are discoverable, but standalone package validation still passes.
Missing optional package	Disabled-but-helpful UI appears; no exceptions, dead buttons, or hidden records.
Narrow window	Header, minimize, help, primary action, filters, and status remain reachable.
Wide window	Panels expand usefully; no large unused gaps where lists/results need space.
Short/tall window	Footer/action bars remain reachable; results/logs/previews get appropriate remaining height.
Overlay tray	Clamps, Escape closes, outside-click behavior works, focus is predictable, and full-tool handoff exists.
Domain reload	Selected tab, splitter widths, filters, tray state, help topic, Developer Mode state, and package simulations persist safely.
Selection changes	Inspector/context/scene surfaces update without stale actions or unexpected mutation.
Scan refresh	Cached state appears first, stale state is clear, refresh cost/scope is visible, and no scan runs during repaint.

33. Authoring and Package Split Review Gates

Authoring Foundation gate

- Does the feature use shared authoring IDs instead of inventing new permanent IDs?
- Does it use shared link targets where possible?
- Does it expose shared metadata, preview, actions, and validation results?
- Does it behave safely when the concrete editor package is missing?
- Does it preserve legacy note data and relationships?
- Does it avoid Core dependencies on concrete editor windows?

Package split gate

- Which package owns this feature?
- Is the feature Core contract, extension workflow, bridge glue, full bundle convenience, or project adapter?
- What dependencies are required?
- What dependencies are optional?
- What capabilities are provided and consumed?
- What extension points are provided and consumed?
- What UI appears when optional packages are missing?
- Can the package compile with only Core plus its declared required dependencies?

- Could this accidentally make the full bundle the only working configuration?

Developer Mode gate

- Does every developer-only control require the root Developer Tools toggle?
- Does every metadata mutation require the metadata mutation gate?
- Does every source-writing tool require the source authoring gate?
- Do direct menu/context/shortcut/tray/overlay actions fail safely when the gate is disabled?
- Are normal user workflows still available without Developer Mode?

Notes and Roadmap migration gate

- Does the pass preserve existing note IDs, metadata, tags, targets, links, and integrations?
- Does it preserve context menus, inspector note summaries, scene badges, help bridges, audit bridges, token links, and documentation-link relationships?
- Does it deprecate only the old freeform writing panel and not the Notes system itself?
- Does it provide read-only selected item preview in the browser?
- Does it route editing to Rich Document Editor when available?
- Does it show safe missing-editor states when unavailable?

Recommended implementation pass sequence

- 38. Developer Mode root-gate hardening across all current developer surfaces.
- 39. Shared window chrome and minimize/tray helper adoption.
- 40. Package metadata descriptor validation and saturation.
- 41. Registry registrar split: move extension registrations out of Core defaults.
- 42. Menu ownership split: move extension menu entries out of Core menu classes.
- 43. Real package presence resolver plus Developer Mode simulation labeling.
- 44. Shared overlay tray manager and tray request model.
- 45. Authoring provider ownership split.
- 46. Asmdef/package skeleton pass.
- 47. Authoring Browser conversion, Rich Document MVP, Board/Graph MVP, Data Sheet MVP.
- 48. Optional bridges in priority order after standalone extensions compile independently.

34. Reference Basis

This edition is based on the current user instruction, the current PungentFunkUtilities source archive supplied in this project, the 2026 package-modularity and Authoring Foundation doctrine, the Overlay Tray doctrine, the Package Split and Utility Extension Map, and the earlier Verifiable UI, Resizable Panel, Shared Systems, and Manual Doctrine editions.

Current source signals used to refine this edition

Source file	Observed signal	Doctrine consequence
Editor/Utilities Core/DeveloperMode/PungentDeveloperMode.cs	Defines Enabled, warning acknowledgement, CanEditProjectMetadata, CanEditPresetSources, and source-authoring availability.	Promote Developer Mode to a root-gated suite contract with visibility, metadata mutation, and source-authoring levels.
Editor/Utilities Core/PungentUtilityControlPanelWindow.cs	Draws Developer Tools in Browser Settings and developer filters/actions only when Developer Mode is enabled.	Make Utilities Browser -> Browser Settings -> Developer Tools the only user-facing root toggle.
Editor/Utilities Core/PungentUtilityDescriptor.cs	Descriptor already carries package ID, tier, dependencies, capabilities, extension points, bridge state, fallback, install hint, documentation topic, visibility, and browser flags.	Make metadata saturation mandatory and validate descriptors before user-facing display.
Editor/Utilities Core/Packages/PungentUtilityPackageCatalog.cs	Maps utility IDs to package records and includes developer override availability; defaults treat many extension packages as installed.	Separate package ownership/catalog metadata from real package presence and Developer Mode simulation.
Editor/Utilities Core/PungentUtilityRegistry.cs	Central registry exposes Register/RegisterRange but still imports concrete lab namespaces and registers concrete extension windows/actions by default.	Move extension registrations into per-package registrars; Core registry owns APIs and fallback discovery only.
BoardGraph, DataSheets, Tokens, Notes, Help source files	Multiple records expose archived/developerOnly/generated/internal	Require metadata mutation gates across all authoring, help, token, board, sheet, and note

Source file	Observed signal	Doctrine consequence
	metadata surfaces.	metadata controls.
Minimizer/tray and utility card source patterns	Core tray/minimizer exists and is already a shared workflow surface.	Promote minimize/tray behavior and tray access to shared chrome contracts.

Limits of this edition

- This document is doctrine, not a snapshot audit.
- It does not prove that every active Unity UI currently complies.
- It intentionally avoids volatile script counts and exhaustive file inventories.
- Implementation passes should still verify active UI, compile state, package matrix, domain reload, and missing-package behavior.

35. Closing Doctrine

PungentFunk Utilities should remain modular, discoverable, and safe. Core owns stable contracts and shared infrastructure. Extensions own concrete workflows. Bridges connect packages without hard dependencies. The full bundle coordinates, but it must not become the only working configuration.

The defining refinement of this edition is consistency by contract. Developer Mode is globally rooted in Utilities Browser settings. Core registry stops owning extension registrations. Every eligible utility uses shared window chrome, help handoff, minimize/tray behavior, package badges, and package fallback UI. Overlay trays are coordinated by one shared manager. Package metadata is complete enough to explain ownership, missing dependencies, and capability states. Every UI claim must be verified through source path, active path, visibility, usability, and validation.

A utility that technically exists but cannot be found, cannot be used, hides normal workflows behind developer toggles, duplicates stale UI, mutates data from repaint, invents local tray behavior, or silently assumes the full bundle is not finished. The product promise is met only when tools are modular, comprehensible, reachable, safe, consistent, and verified.